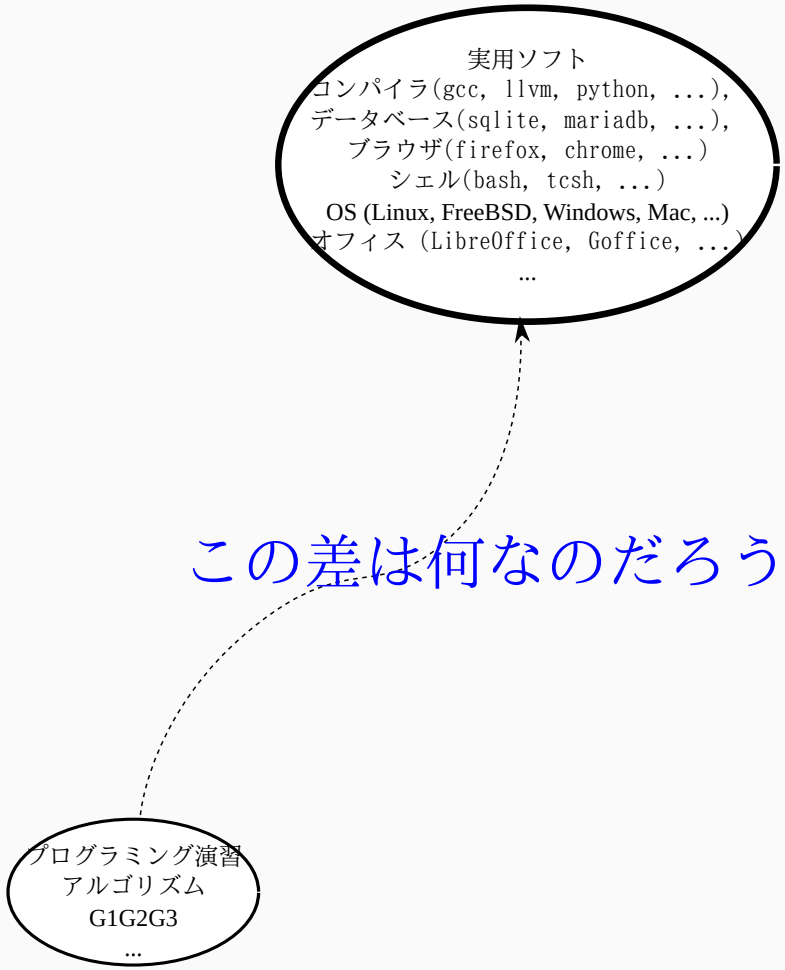
# 大規模ソフトウェアを手探る

田浦 健次朗
前田 優希 (田浦研 M1), 酒井 元 (田浦研 B4)

2026/10/06

# この課題の目標

- 「演習レベルの小さなプログラムが作れること」と,「実用規模のプログラムが作れること」のギャップを埋める(ための知識と経験を得る)
- ささやかでも, ソフトをどう改良するかについて, アイデアを出しあう(what と how)



# ぜひマスターしてほしいスキル

- 全容がよくわからないソフトウェアの概要を把握し, 拡張・変更できるようになる
- ← そのために, ソースファイル中で修正が必要な場所を突き止める
- ← そのためのツール
  - 汎用的な文字列検索(grep など)
  - クロスリファレンスツール(globals など)
  - デバッガ ← 個人的にはこれが最重要と考えている
  - プロファイラ, トレーサ (uftrace)
  - AI (スキルなのかはともかく)

# 代表的なプログラミング言語とビルドの流儀

- C/C++
  - configure; make; make install
  - cmake; make; make install
- Python
  - pip
- JavaScript (Node.js)
  - npm
- Rust, Go, …
- ソースコードのダウンロード → ビルド → デバッガなどで追跡 → 修正 → 反映 の流れをマスター

# これを身につけることの意義

- ソフトウェアの研究では, 成果をソフトウェアとして発信することが重要
- しかし大きなソフトウェアを一から作ることは, 困難な場合が多い
- → 既存のソフトウェアの拡張として作ることが多い
- 一から作る場合でも, 成果を実用的なソフトウェアとして発信する際, 常識的なお作法を守っておくことは重要

# 課題期間全体のロードマップ (目安)

- 1 日目 : ソフトのインストール, デバッガで動作追跡手法を練習
- 2-3 日目 : チームを作る, 手探るソフト, 目標議論スタート
- 3-4 日目 : 試しになんでもいいから変更してみる
- 4-9 日目 : 目標を決め, 実行
- 10 日目 : 最終発表

# 日々行うこと

- 議論
- 作業
- その記録 (進捗記録)
  - 他の人, あるいは後の自分が一から再現できるような情報を記録
  - トラブルも進捗
    - 他の人が再現できる情報
    - なるべく小さな例で (Minimum Working Example)
- 進捗発表 (チーム結成以降, 各回終了時, 持ち回りで何班かずつ)
- 他の班の進捗発表を見て, 質問, コメント, 互助

# 本日の以降の作業

- 新テキスト を眺める
- そこにある「コードのダウンロード, ビルド, デバッグの練習」をやってみる
  - C/C++ 編
  - Python 編
  - JavaScript 編
- (テキストには書いていないが) AI がどれだけヒントをくれるかもやってみるとよい

# AI 利用について

- Coding Agent (OpenCode) を提供
- コードの全体像分析, 直すべき箇所の把握などに使う
- 使い方, それ以上の精神論はホームページに記載した